

CSE_3523

Segment 3:Part-2
Instruction Set

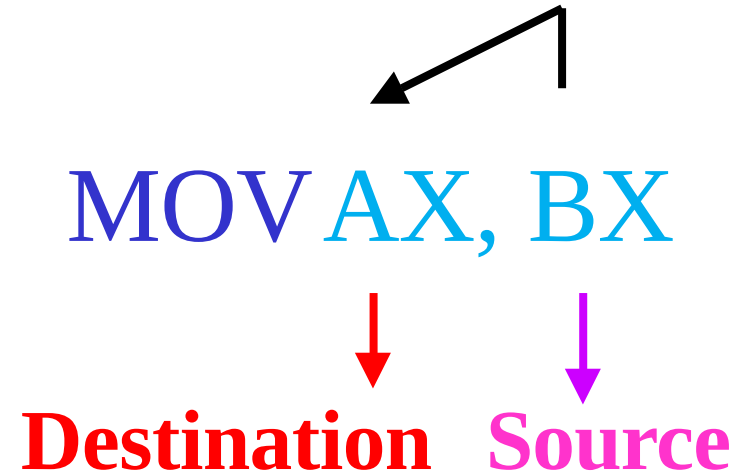
Text Book

Topic: 3.3,4.1,4.2,4.5,5.1,5.2,5.4,5.5,6.5

Barry B. Brey, **The Intel Microprocessors**

Data Transfer Instructions
MOV, XCHG, PUSH, POP

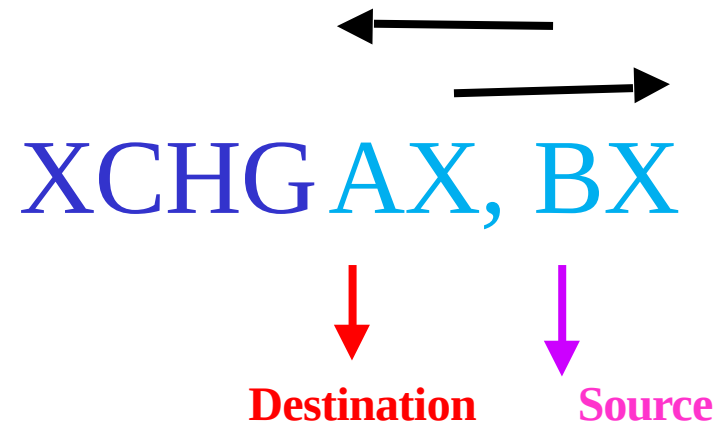
MOV



MOV AX, BX instruction translates the word contents of the **source** register (BX) into the **destination** register (AX).

- Example :
- MOV AX, BX
- MOV AX, 2150H
- MOV AL, [1135]

MOV



This (Exchange) instruction **switches** the contents of the source and destination operands.

$(\text{Temp}) \leftarrow (\text{DEST})$
 $(\text{DEST}) \leftarrow (\text{SRC})$
 $(\text{SRC}) \leftarrow (\text{Temp})$

Example :

XCHG AX, BX
XCHG BL, AL

PUSH-POP(Stack Memory-Addressing)

- ❑ The stack plays an important role in all MP.
- ❑ It holds data temporarily & store return addresses for procedures.
- ❑ Stack memory is a LIFO (Last-in-First-out) memory, which describes the way that data are stored & removed from the stack.
- ❑ Data placed onto stack with a PUSH instruction & removed with a POP instruction.

PUSH-POP(Stack Memory-Addressing)

- ❑ Whenever a word of data is PUSHed onto the stack,
 - the high-order 8-bits are placed in the location addressed by $(SP - 1)$.
 - Low-order 8-bits are placed in the location addressed by $(SP - 2)$.
 - SP is then decremented by 2 so that next word of data is stored in the next available stack memory location.
- ❑ Whenever data POPped from the stack,
 - low-order 8-bits are removed from the location addressed by SP.
 - High-order 8-bits are removed from the location addressed by $(SP + 1)$.
 - SP register is then incremented by 2.
- ❖ PUSH & POP always store or retrieve words of data, never bytes in 8086.

PUSH-POP(Stack Memory-Addressing)

1. PUSH A

2. POP A

3. PUSH F

4. POP F

5. PUSH WORD PTR [BX]

6. PUSH CX

7. POP CS

8. POP AH

1. copies AX, CX, DX, BX, SP, BP, SI, DI to the stack.
2. Removes word contents for following registers from the stack: DI, SI, BP, SP, BX, DX, CX, AX.
3. copies the flag register to the stack.
4. removes a word size and places it into flag register.
5. copies word contents of the data segment memory location addressed by BX onto the stack.
6. copies the CX register to the stack.
7. instruction is wrong.
8. instruction is wrong.

PUSH-POP(Stack Memory-Addressing)

MOV AX, 5566H

MOV BX, 7788H

MOV SP, 2009H

PUSH AX

PUSH BX

POP CX

POP DX

CX=7788H

DX=5566H

SS		
		2009 SP ₀
55		2008
66		2007 SP ₁
77		2006
88		2005 SP ₂
		2004
		2003
		2002

$$SP_{\text{PUSH}} = SP_0 - 2n$$

$$SP_{\text{POP}} = SP_0 + 2m$$

$$SP_{\text{final}} = SP_0 - 2n + 2m$$

PUSH-POP(Stack Memory-Addressing)

Given, AX=7788H and BX=99AAH. Interchange the value of AX and BX using PUSH and POP operation.

```
MOV AX, 7788H
```

```
MOV BX, 99AAH
```

```
PUSH AX
```

```
PUSH BX
```

```
POP AX
```

```
POP BX
```

Arithmetic Instructions

**ADD, INC ADC, SUB,
DEC, SBB, MUL, DIV**

ADDITION

- ❑ Addition (**ADD**) appears in many forms in the microprocessor.
- ❑ A second form of addition, called add-with-carry, is introduced with the **ADC** instruction.
- ❑ Increment instruction (**INC**) is a special type of addition that **adds 1** to a number.
- ❑ When **arithmetic and logic instructions execute**, **contents of the flag register change**.
 - I (interrupt), T (trap), and D (direction) flags do not change.
- ❑ Any ADD instruction modifies the contents of the C (carry), A (auxiliary carry), P (parity), Z (zero), S (sign), and O (overflow) flags.

ADDITION

- ❑ **ADD AL, BL** – instruction performs $AL=AL+BL$
- ❑ **ADD AX, BX** – instruction performs $AX=AX+BX$
- ❑ **ADD [BX], AL** – AL adds to the byte contents of the DS memory location addressed by BX with the sum stored in the same memory location.

ADDITION

MOV DL, 12H

ADD DL, 33H

12H = 0001 0010

33H = 0011 0011

45H = 0100 0101

After the addition, the sum (45H) moves into DL and the flag change, as follows:

C=0 (no carry)

A=0 (no auxiliary carry)

P=0 (odd parity)

Z=0 (result not zero)

S=0 (result positive)

O=0 (no overflow)

T=1 I=0 D=0

ADDITION

Perform and write the change in flags.

```
MOV AX, 30F0H
MOV BX, 2000H
ADD AX, WORD PTR[BX]
```

DS memory map

3C	2000
FF	2001
0F	2002
FE	2003

30F0H = 0011 0000 1111 0000

FF3CH = 1111 1111 0011 1100

302CH = 1 0011 0000 0010 1100

BX=302CH and C, A, P, Z, S, O ??

```
MOV AX, 30F0H
MOV BX, 2000H
ADD [BX], AL
```

Draw memory and write flags change

DS memory map

3C 2C	2000
FF	2001
0F	2002
FE	2003

ADDITION with Carry

- ❑ ADC instruction adds the bit in the carry flag (C) to the operand data.
- ❑ Like ADD, ADC affects the flags after the addition.
- ❑ e.g., **ADC AL, AH** – instruction performs $AL = AL + AH + \text{carry}$
- ❑ **ADC DH, [BX]** – the byte contents of the DS memory location addressed by BX add to DH & carry, with the sum stored in DH.

ADDITION with Carry

- ❑ Write an assembly language program to add two 16 bit Hex numbers **20F1H and 3C72H** using using 8 bit Addition . Also Draw register and write flags change after final operation

```
MOV AX, 20F1H
MOV BX, 3C72H
ADD AL,BL
ADC  AH,BH
```

ADDITION with Carry

- ❑ Write an assembly language program to add five 16 bit Hex numbers.

Put them in DS memory SI=2004H.

```
MOV SI, 2004H
MOV [SI], 6566H
MOV AX, [SI]
INC SI
INC SI
MOV [SI], 8876H
ADD AX, [SI]
INC SI
INC SI
```

```
MOV [SI], 7755H
ADC AX, [SI]
INC SI
INC SI
MOV [SI], 5566H
ADC AX, [SI]
INC SI
INC SI
MOV [SI], 6677H
ADC AX, [SI]
```

ADDITION with Carry

- ❑ Write an assembly language program to add five 16 bit Hex numbers. Put them in DS memory SI= 2004H,

DS memory map

66	2000
65	2001
76	2002
88	2003
55	2004
77	2005
66	2006
55	2007
77	2008
66	2009
	2010

Subtraction

- ❑ Many forms of subtraction (**SUB**) appear in the instruction set.
- ❑ A special form of subtraction (decrement or **DEC**) subtracts 1 from any register or memory location.
- ❑ Another type subtraction is the subtract-with-borrow instruction (**SBB**).
- ❑ After each subtraction, the microprocessor modifies the contents of the flag register.
 - ❑ **SUB CL, BL** – instruction performs $CL = CL - BL$
 - ❑ **SUB AX, SP** – instruction performs $AX = AX - SP$
 - ❑ **SUB [DI], CH** – subtracts CH from the byte contents of the DS memory location addressed by DI and stores the difference in the same memory location.

Subtraction₁

MOV AL, 22H
SUB AL, 44H

22H
-44H

DEH

¹ 0010 0010
- 0100 0100

1101 1110

2's complements.

0010 0001
+1

0010 0010
2 2

DEH is the
negative Number
Representation
of 22

DE=-22

After the subtraction, the results
(-22H) moves into AL and the
flag change, as follows:

C=1 (borrow)

A=1 (auxiliary borrow)

P=1 (even parity)

Z=0 (result not zero)

S=1 (result negative)

O=0 (no overflow) Overflow occurs if signed result
greater than +127 or less than -128.

Decrement Subtraction

- ❑ Decrement Subtraction (**DEC**) instruction subtracts '1' from a register/memory location.
- ❑ e.g., **DEC BH** – instruction performs $BH = BH - 1$
- ❑ **DEC BYTE PTR[BX]** – subtracts '1' from the word contents of the DS memory location addressed by BX.

MOV DI, 2003H
DEC BYTE PTR[DI]

$$\begin{array}{r} 14 \\ -1 \\ \hline 13 \end{array}$$

DS memory map

11	2000
12	2001
13	2002
14 13	2003
15	2004

Subtraction-with-Borrow (SBB)

- ❑ A subtraction-with-borrow (SBB) instruction functions as a regular subtraction, except that the carry flag (C), which holds the borrow, also subtracts from the difference.
- ❑ Like SUB, SBB affects the flags after the subtraction.
- ❑ e.g., **SBB AH, AL** – instruction performs $AH = AH - AL - \text{carry}$
- ❑ **SBB [DI], AL** – both AL and carry subtract from DS memory location addressed by DI and result stored in the same location.

Multiplication

- ❑ Multiplication is performed on bytes or words, and can be signed (IMUL) or unsigned integer (MUL).
- ❑ Product after a multiplication always a double-width product.
 - Two 8-bit numbers multiplication generate a 16-bit product.
 - Two 16-bit numbers generate a 32-bit.
- ❑ With 8-bit multiplication, the multiplicand is always in the AL register, whether signed or unsigned.
- ❑ e.g., **MUL CL** – AL is multiplied by CL; the unsigned product is in AX. msb 0, carry zero
- ❑ **IMUL DH** – AL is multiplied by DH; the signed product is in AX.

```
MOV AX, 5566H  
MOV BX, 7788H  
MUL BL
```


Multiplication

- ❑ 16-bit multiplication is very similar to byte multiplication.
- ❑ AX contains the multiplicand instead of AL and 32-bit product appears in DX–AX instead of AX.
- ❑ The DX register always contains the most significant 16 bits of the product; AX contains the least significant 16 bits.
- ❑ e.g., **MUL CX** – AX is multiplied by CX; the unsigned product is in DX-AX.
- ❑ **IMUL DI** – AX is multiplied by DI; the signed product is in DX-AX.

MOV AX, 5566H

MOV BX, 7788H

MUL BX

Multiplication

- ❑ Write assembly language for the equation $y = 5 \times 6 \times 10H$.

```
MOV AL, 5H
```

```
MOV BL, 6H
```

```
MUL BL
```

Result will go to AX (16 bit)

```
MOV BX, 10H
```

```
MUL BX
```

Result will go to DX-AX (32 bit)

Division

- ❑ Occurs on 8- and 16-bit numbers depending on microprocessor.
 - These numbers are signed (IDIV) or unsigned (DIV) integers.
- ❑ An 8-bit division divides a 16-bit number by an 8-bit number; a 16-bit division divides a 32-bit number by an 16-bit number.
- ❑ None of the flag bits change predictable in division operation.
- ❑ 8-bit division uses AX to store the dividend that is divided by the contents of any 8-bit register or memory location.
- ❑ Quotient moves into AL after the division with AH containing a whole number remainder.
- ❑ e.g., **DIV CL (IDIV BL)** – AX is divided by CL; the unsigned quotient is in AL and the unsigned remainder is in AH.

Logical Instructions

AND, OR, XOR, , SHR, SHL

Basic Logic Instructions

- ❑ Include AND, OR, Exclusive-OR, NOT, NEG, Shift, Rotate, TEST.
- ❑ All logic instructions affect the flag bits. (i.e.
- ❑ Logic operations always clear the carry and overflow flags (i.e. C=O=0), other flags will change except T, I, D flags.

MSB

LSB

7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---



Bit Position

8	7	6	5	4	3	2	1
---	---	---	---	---	---	---	---



Bit Number

Basic Logic Instructions-(AND)

- ❑ Performs logical multiplication, illustrated by a truth table.
- ❑ AND **clears** bits of a binary number.
 - The task of clearing a bit in a binary number is called **Masking**.
- ❑ AND uses any mode except memory-to-memory and segment register addressing.



Logic symbol

A	B	T
0	0	0
0	1	0
1	0	0
1	1	1

Truth table

Basic Logic Instructions-(AND)

x x x x x x x x Unknown number

0 0 0 0 1 1 1 1 Mask

0 0 0 0 x x x x Result

Leftmost 4-bits
masked/cleared

❑ e.g., `AND AL, BL` – $AL = AL \text{ AND } BL$

❑ `AND AX, [DI]` – the word contents of DS memory location addressed by DI are *ANDed* with AX and result stored in AX.

Basic Logic Instructions-(AND)

- ❑ Given a number BBH, then write an instruction to Clear leftmost 2-bits and rightmost 2-bits.

MOV AL, BBH
AND AL, 3CH

BBH	1011 1011
3CH	0011 1100
38H	0011 1000

Clear bit multiply by
0 and others put 1

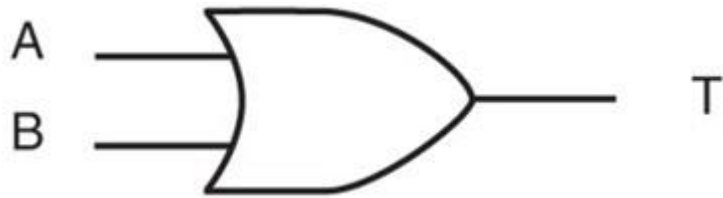
- ❑ If CX=FF0EH, write an instruction to Clear the bit position 5, 7 & 8th.

MOV CX, FF0EH
AND CX, FE5FH

FF0EH	1111 1111 0000 1110
FE5FH	1111 1110 0101 1111
FE0EH	1111 1110 0000 1110

Basic Logic Instructions-(OR)

- ❑ Performs logical addition.
- ❑ OR **sets** any bit of a binary number.
- ❑ The OR instruction uses any addressing mode except segment register addressing.



Logic symbol

A	B	T
0	0	0
0	1	1
1	0	1
1	1	1

Truth table

Basic Logic Instructions-(OR)

	x x x x	x x x x	Unknown number
+	0 0 0 0	1 1 1 1	Mask
	x x x x	1 1 1 1	Result

Rightmost 4-bits set

❑ e.g., `OR AL, BL` – `AL = AL OR BL`

❑ `OR DX, [BX]` – `DX` is *ORed* with the word contents of `DS` memory location addressed by `BX` and result stored in `DX`.

Basic Logic Instructions-(OR)

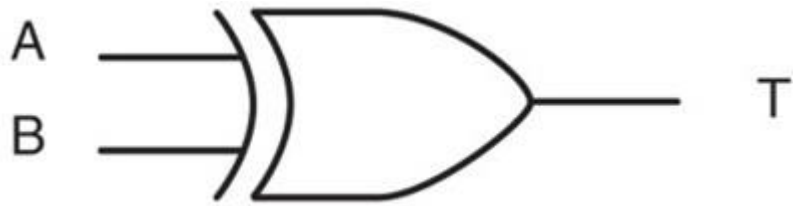
- Write an assembly language program to set rightmost 2-bits and to clear leftmost 3-bits of register value AX=FF0EH.

```
MOV AX, FF0EH  
OR  AX, 0003H  
AND AX, 1FFFH
```

	FF0EH	1111	1111	0000	1110
OR	0003H	0000	0000	0000	0011
		1111	1111	0000	1111
AND	1FFFH	0001	1111	1111	1111
	1F0FH	0001	1111	0000	1111

Basic Logic Instructions-(X-OR)

- ❑ XOR is used to **inverts** any bit of a binary number.
- ❑ XOR uses any addressing mode except segment register addressing.
- ❑ A common use for the Exclusive-OR instruction is to clear a register to zero.



Logic symbol

A	B	T
0	0	0
0	1	1
1	0	1
1	1	0

Truth table

Basic Logic Instructions-(X-OR)

	x	x	x	x	x	x	x	Unknown number
⊕	0	0	0	0	1	1	1	Mask
	x	x	x	x	$\bar{x}$	$\bar{x}$	$\bar{x}$	Result

Rightmost 4-bits inverted

□ e.g., `XOR CH, DL` – CH = CH XOR DL

□ `XOR DX, [SI]` – DX is *XORed* with the word contents of DS memory location addressed by SI and result stored in DX.

Basic Logic Instructions-(X-OR)

- ❑ Write an instruction to invert the leftmost 6-bits of AX register. Given, AX=FF0EH.

```
MOV AX, FF0EH  
XOR AX, FC00H
```

FF0EH	1111	1111	0000	1110
FC00H	1111	1100	0000	0000
030EH	0000	0011	0000	1110

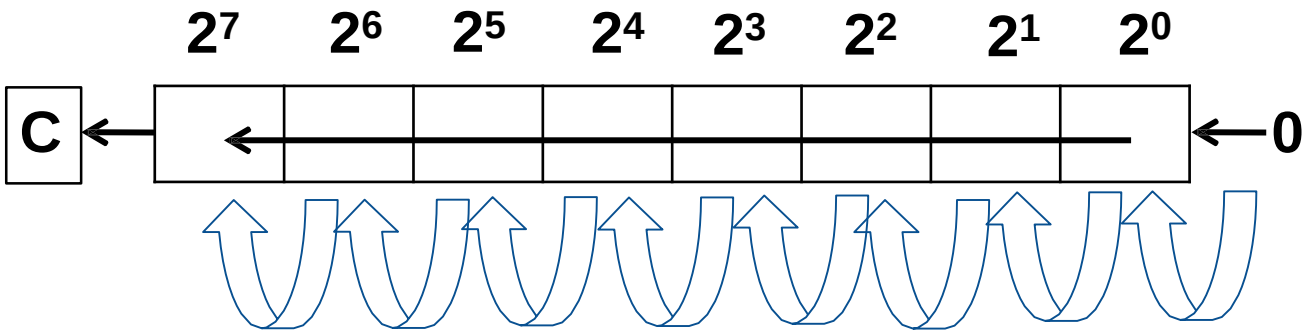
Same bit: XOR 0

SHIFT

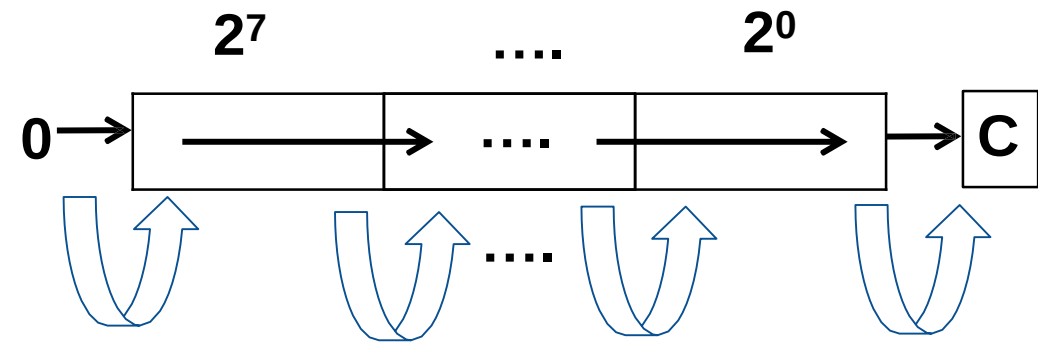
- ❑ Position or move numbers to the left or right within a register or memory location.
 - Also perform simple arithmetic as multiplication by powers of 2^{+n} (left shift) and division by powers of 2^{-n} (right shift).
- ❑ The microprocessor's instruction set contains four different shift instructions:
 - Two logical shifts: Logical shift Left (**SHL**) & Logical shift Right (**SHR**).
 - Two arithmetic shifts: Arithmetic shift Left (**SAL**) & Arithmetic shift Right (**SAR**).

SHIFT

- ❑ Logical shifts multiply or divide unsigned data; arithmetic shifts multiply or divide signed data.



Multiplication

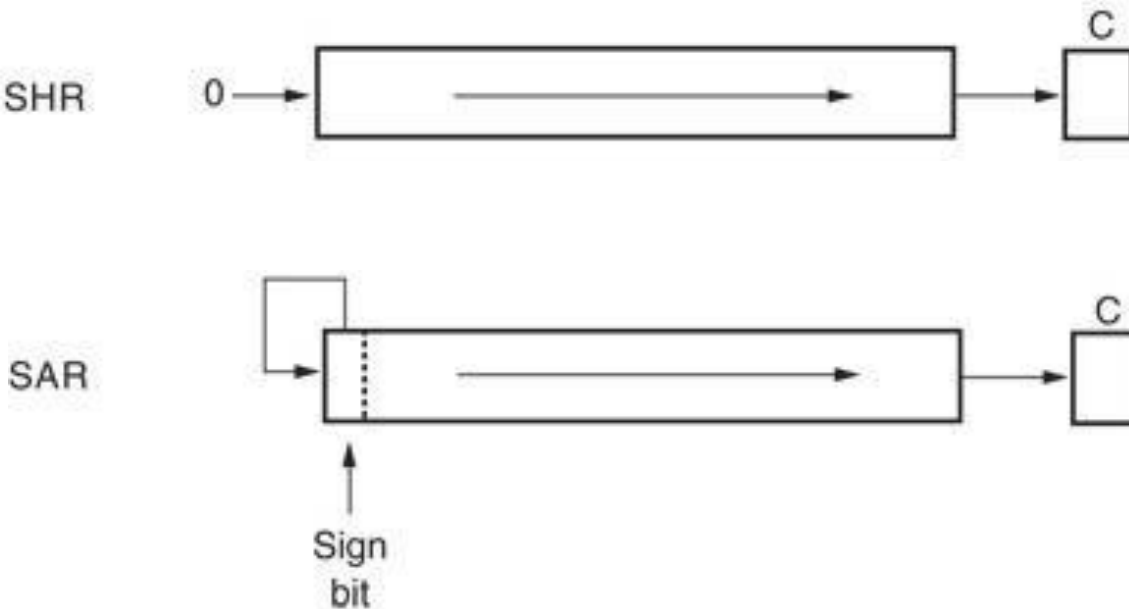
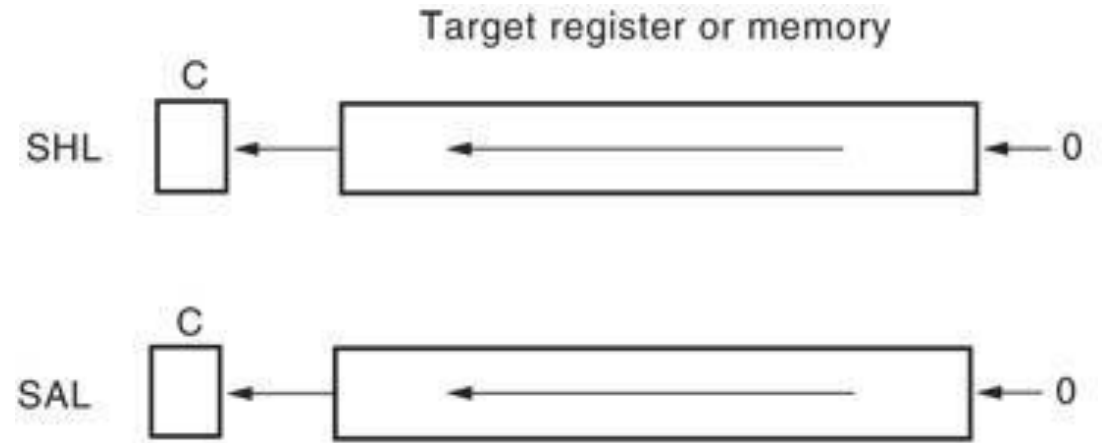


Division

- ❑ e.g., **SHL AX, 1** – AX is logically shifted left 1 place.
- ❑ **SAR SI, 2** – SI is arithmetically shifted right 2 places.

SHIFT

- Move 0 in the rightmost bit for a logical left shift;
- Move 0 in the rightmost bit for a arithmetic left shift;

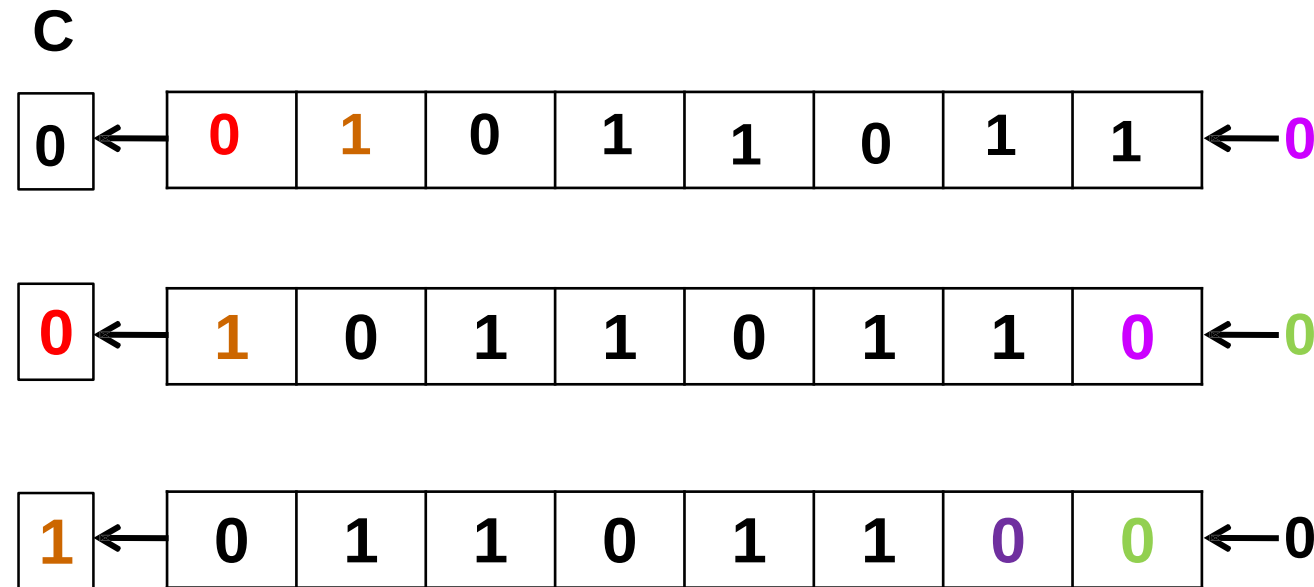


- Move 0 to the leftmost bit position for a logical right shift;
- Arithmetic right shift copies the sign-bit through the number.

SHIFT

MOV AL, 5BH
SHL AL, 2

What will be the final value of register and carry?



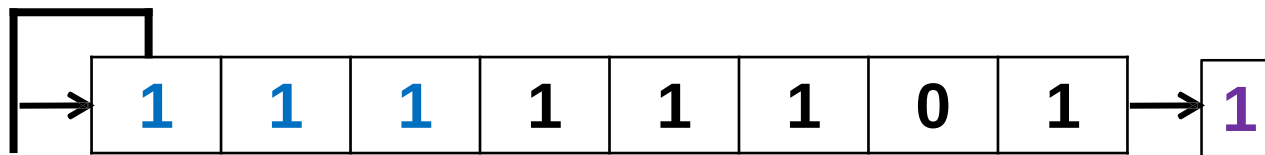
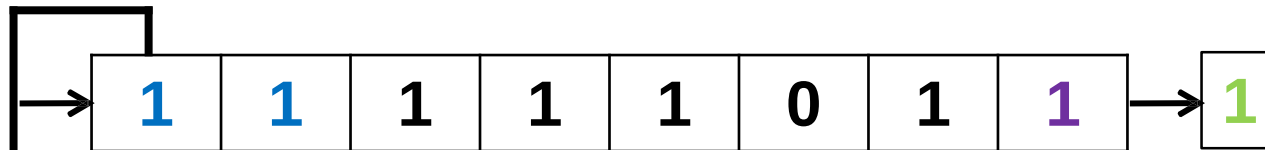
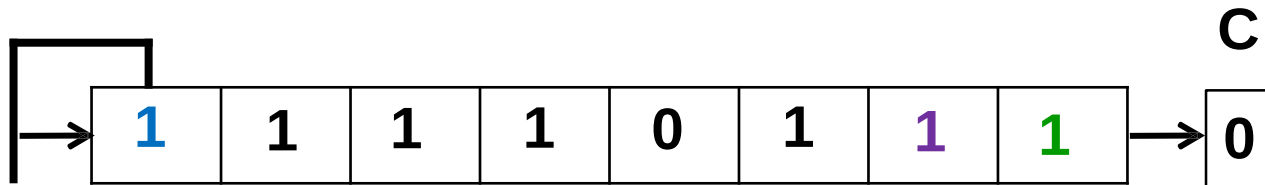
MOV CL, 2H
MOV AL, 5BH
SHL AL, CL

Answer: the final value of register is 6CH and carry is 1.

SHIFT

MOV AL, F7H
SAR AL, 2

What will be the final value of register and carry?



MOV AX, F755H
SAR AX, 8

Answer: the final value of register is FDH and carry is 1.

Processor Control Instructions

Mnemonics	Explanation
STC	Set CF $\leftarrow 1$
CLC	Clear CF $\leftarrow 0$
CMC	Complement carry CF $\leftarrow \text{CF}'$
STD	Set direction flag DF $\leftarrow 1$
CLD	Clear direction flag DF $\leftarrow 0$
STI	Set interrupt enable flag IF $\leftarrow 1$
CLI	Clear interrupt enable flag IF $\leftarrow 0$
NOP	No operation
HLT	Halt after interrupt is set
WAIT	Wait for TEST pin active
ESC opcode mem/ reg	Used to pass instruction to a coprocessor which shares the address and data bus with the 8086
LOCK	Lock bus during next instruction

CT Syllabus

Segment 3: Stack Addressing Mode,
Addition,ADC,subtraction,
Basic Logic instructions(AND,OR,X-OR,SHIFT)